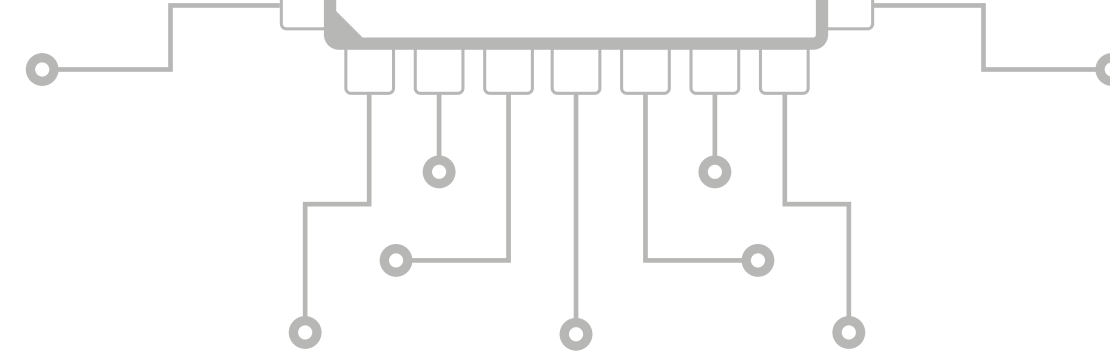
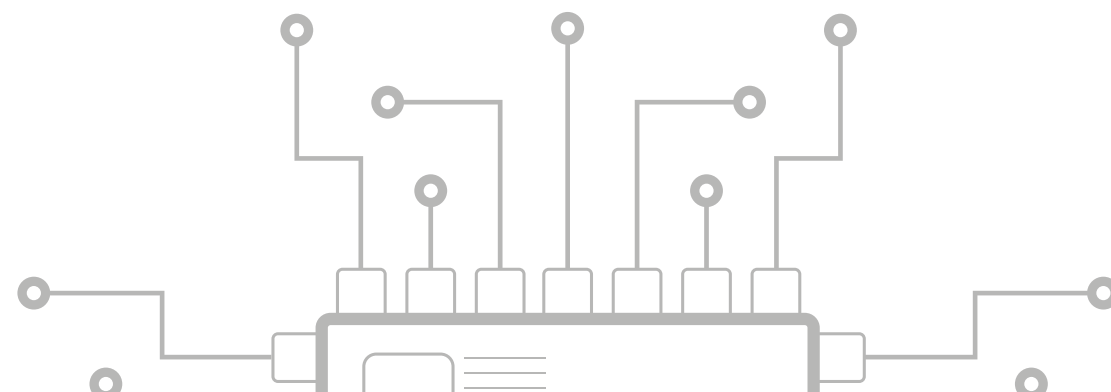




Proměnné a datové typy v C

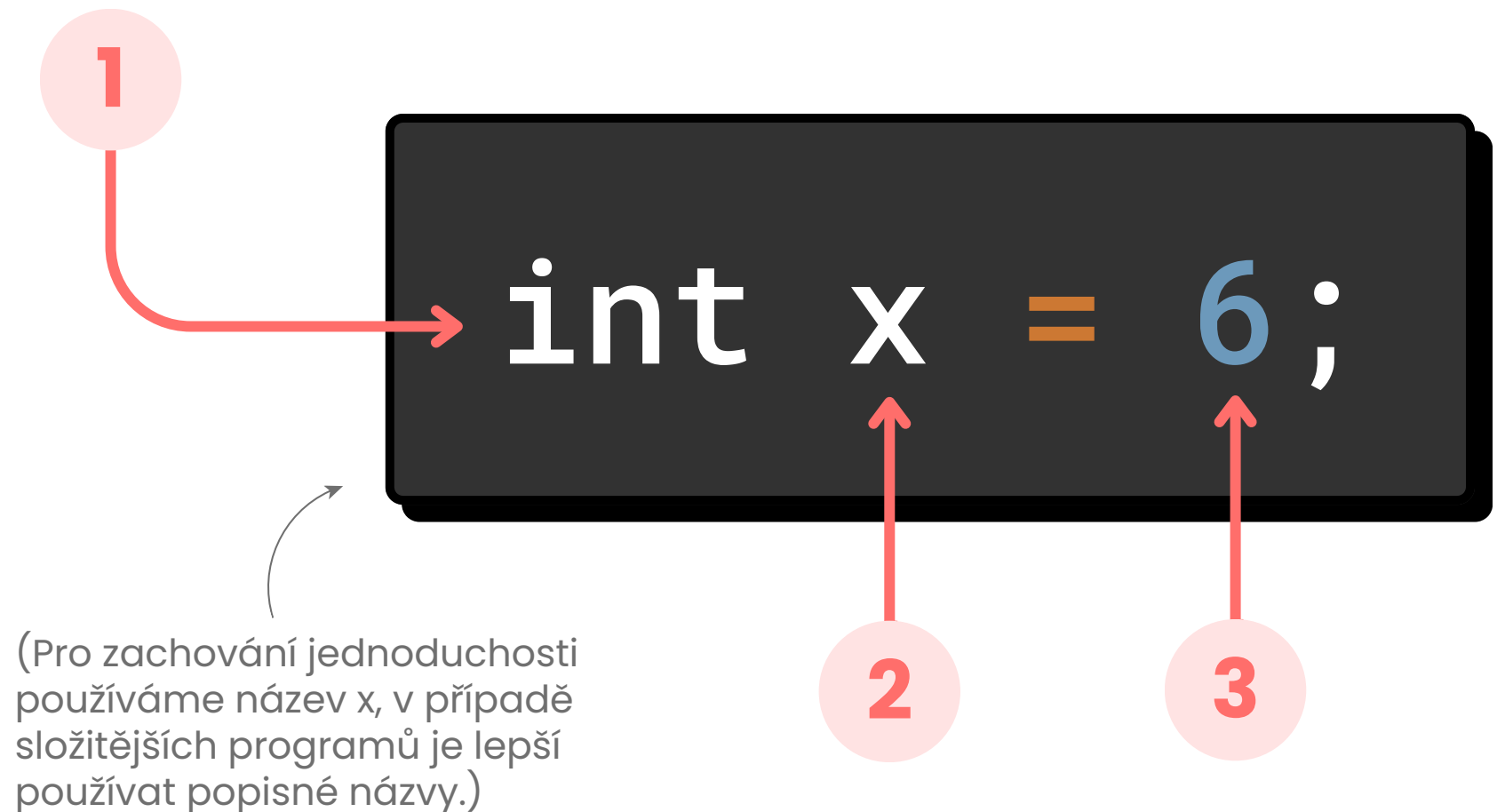
Proměnné, datové typy, typové systémy,
formátovací značky, konstanty

Dominik Němeček



Proměnná

- Proměnné **slouží k ukládání hodnot**, které lze v průběhu programu měnit a zpracovávat.
- Lze si to představit jako **nádoby, do kterých lze uložit různá data**, například čísla, texty nebo znaky.



1

Datový typ

Definovaný datový typ, který určuje, **jaký druh dat proměnná obsahuje** (text, číslo, znak, ...).

2

Název proměnné

Proměnná musí mít **název, který ji identifikuje**. Název by měl být popisný, aby bylo zřejmé, jaký typ dat proměnná uchovává.

3

Hodnota

Proměnná **by měla mít také hodnotu**, ale může ze začátku být i bez hodnoty.

Datové typy

- Datový typ v programování určuje, **jaký druh dat může proměnná obsahovat a co s nimi můžeme dělat**.
- Říká nám, zda je proměnná například celé číslo, písmeno, nebo desetinné číslo, a **kolik místa zabere v paměti počítače**.
- Každý **datový typ má svoje pravidla** pro práci s hodnotami, které obsahuje.

Skupina datových typů	Datové typy
Základní typy	int, char, float, double, short, long, bool
Odvozené typy	pointer, array
Uživatelsky definované typy	union, typedef, enum, struct
Znamínkové a neznamínkové	signed, unsigned

Základní typy

(Odvíjí se od typu systému – Win/Linux, 32bit/64bit systém.)

(Používá se při výpisu a ukládání hodnoty v proměnné).

Datový typ	Hodnota	Velikost	Formátovací značka	Rozsah
int	Celé číslo	4 byte (32 bit)	%d	-2,147,483,648 až 2,147,483,647
char	Znak/celé číslo/text	1 byte (8 bit)	%c pro znak, %s pro text	-128 až 127
short	Celé číslo	2 byte (16 bit)	%hd	-32,768 až 32,767
float	Desetinné číslo	4 byte (32 bit)	%f, %.2f – nastavení des. míst	1.2E-38 až 3.4E+38
double	Desetinné číslo	8 byte (64 bit)	%lf	1.7E-308 až 1.7E+308
long	Celé číslo	8 byte (64 bit)	%ld	-9.2E+18 až 9.2E+18
_Bool	True/false	1 byte (8 bit)	%d	0 až 1

Deklarace proměnné

- Proces, při kterém uvádíme **datový typ a název proměnné**.
- Dáváme informaci o tom, **jaký druh dat bude proměnná uchovávat**.
- Slouží k tomu, **aby kompilátor věděl, jaký paměťový prostor alokovat**.

1

Deklarace int

Deklarování proměnné pro **uchování celých čísel**.

2

Deklarace float

Deklarování proměnné pro **uchování desetinných čísel**.

```
int x;  
float y;
```

(Při deklaraci nedáváme informaci o konkrétní hodnotě v proměnné, ale pouze o jaký datový typ se jedná).

(Pro používání proměnné je potřeba proměnnou vždy deklarovat).

Inicializace proměnné

- Proces, při kterém uvádíme **datový typ a název proměnné a zároveň nastavujeme vnitřní hodnotu**.
- Dáváme informaci o tom, **jaká hodnota se bude v proměnné nacházet**.

```
int x = 6;  
float x = 7.2f;
```

(Při inicializaci dáváme informaci o konkrétní hodnotě v proměnné).

(Pro používání proměnné není potřeba proměnnou vždy inicializovat, stačí pouze deklarovat).

1

Inicializace int

Inicializace proměnné pro **uchování celých čísel**.

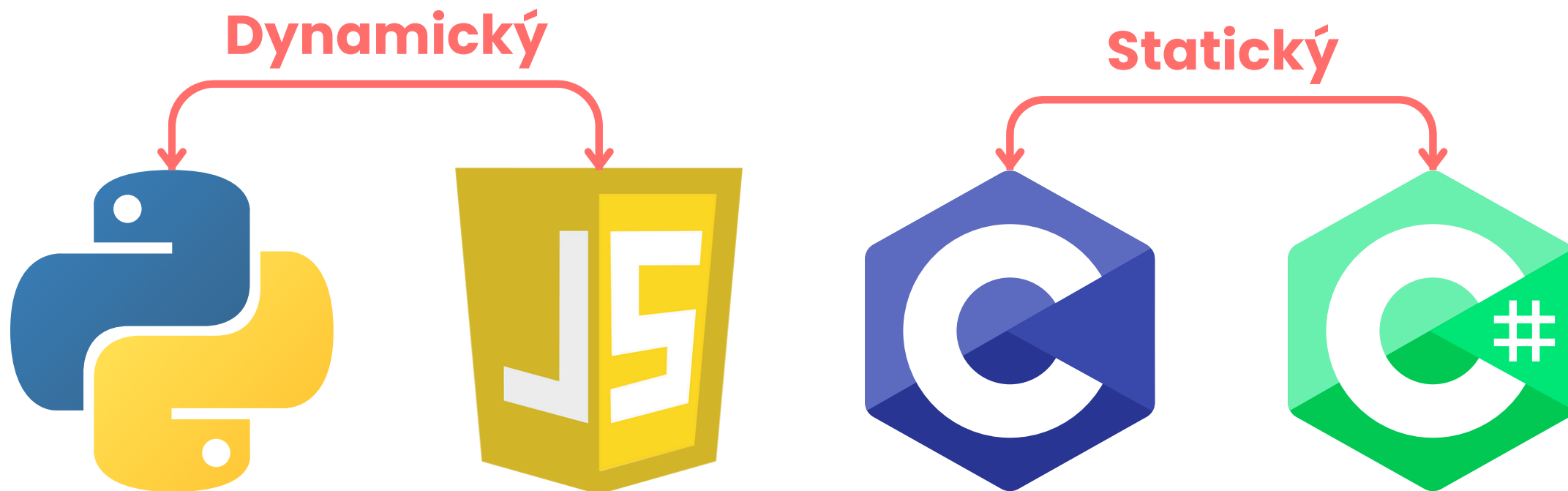
2

Inicializace float

Inicializace proměnné pro **uchování desetinných čísel**.

Typový systém

- **Statický** = Vyžaduje definovat typ proměnné a ten je poté neměnný. Více objemný kód, lepší kontrola.
- **Dynamický** = Není potřeba definovat typ ani deklarovat proměnnou. Systém si sám přiřadí datový typ a proměnnou deklaruje. Méně objemný kód, horší kontrola.
- C je staticky typovaný jazyk.



Životnost proměnné

- Globální proměnná = Proměnná, která je deklarována na začátku programu, mimo jakékoliv funkce nebo bloky kódu.
- Lokální proměnná = Proměnná, která je deklarována a používána pouze v rámci určitého omezeného rozsahu programu.

Lokální

```
#include <stdio.h>

int main()
{
    int x = 10;
}
```

Globální

```
#include <stdio.h>
int x = 10;

int main()
{
}
```

Znamínkové a neznamínkové typy

- Slouží k **upřesnění rozsahu čísel**, které můžeme uložit do jednotlivých datových typů.
- **Signed** = Při použití můžeme ukládat kladná i záporná čísla (defaultně daný, nemusíme psát).
- **Unsigned** = Při použití můžeme ukládat pouze kladná čísla.

```
Minimální hodnota pro signed int: -2147483648  
Maximální hodnota pro signed int: 2147483647  
Minimální hodnota pro unsigned int: 0  
Maximální hodnota pro unsigned int: 4294967295
```

```
unsigned int x = 10;  
signed int x = 10;
```

(Klíčové slovo signed není potřeba specifikovat, protože každý datový typ se automaticky chová jako signed).

(Při použití unsigned můžeme využít dvojnásobnou kapacitu datového typu, aniž bychom zabírali více místa v paměti).

Pojmenování proměnných

1

Srozumitelnost

Proměnná by měla být pojmenována tak, aby bylo jasné, co je v ní uloženo.

Správně

✓ výsledek, výpočet

Špatně

✗ promenna

2

Bez diakritiky

Proměnná by měla být bez diakritiky, přičemž můžeme využívat CZ nebo EN.

Správně

✓ result, výpočet

Špatně

✗ výsledek, výpočet

3

Bez mezer

Proměnná by měla být bez mezer a speciálních znaků, přičemž místo mezery použijeme (_).

Správně

✓ pocet_zprav, pocetAut

Špatně

✗ pocet zprav, pocet Aut

4

Bez zkratek

Proměnná by měla být pojmenována smysluplně a bez zkratek.

Správně

✓ pocet_zprav, zprava

Špatně

✗ pz, zprv

(U jednodušších programů se dají názvy typu x, y omluvit, je ale potřeba vždy dodržovat řádnou dokumentaci.)

(Zde je potřeba upozornit, že bychom si vždy měli vybrat jeden jazyk, nikoliv kombinovat dva jazyky dohromady.)

(Zde je potřeba upozornit, že bychom si vždy měli vybrat jeden způsob, nikoliv kombinovat dva způsoby pojmenování dohromady.)

(U jednodušších programů lze toto tolerovat za předpokladu, že vše ostatní je správně dokumentováno.)

Výpis hodnoty proměnné

- Zkusíme si inicializovat globální proměnnou typu int s hodnotou 8.
- Poté hodnotu proměnné vypíšeme pomocí **funkce printf()**.

```
#include <stdio.h>
int x = 8;

int main()
{
    printf("%d", x);
}
```

1

1

Formátovací značka

Pokud chceme vypsat obsah proměnné x, musíme použít tzv. **formátovací značku (pro datový typ int to je %d)**.

Za tuto formátovací značku se následně **dosadí naše hodnota, která je uložena v proměnné x.**

(Pozor na častou chybu, když se snažíte vypsat obsah proměnné a zvolíte formátovací značku, ale následně zapomenete napsat název proměnné, z které chcete čerpat hodnotu.)

01 Ukázka – výpis součtu

- Inicializujeme si **dvě proměnné typu float** s čísly 2,1 a 5,6.
- Sečteme a vypíšeme výsledek, který zaokrouhlíme na 2 desetinná místa.

Zdrojový kód: 01_Promenne_Prezentace

```
#include <stdio.h>
```

```
float x = 2.1f;
```

```
float y = 5.6f;
```

```
int main()
```

```
{
```

```
    printf("Výsledek je: %.2f", x + y);
```

```
}
```

1

Desetinné místo

Pomocí tohoto formátu lze zaokrouhlit výpis čísel na dvě desetinná místa.

2

Aritmetická operace

Při výpisu proměnné lze používat typické aritmetické operace (+, -, /, *).



Konstantní proměnné

- Konstantní proměnné jsou tzv. neměnné proměnné tzn. nemůžeme přepisovat hodnotu uvnitř.
- Využíváme dva způsoby **#define** a **const**.

(Konstanta `const` funguje prakticky stejně jako `#define`, liší se akorát z hlediska překladač a názvu proměnné tzn. název konstanty pomocí `#define` se vždy píše velkými písmeny.)

Zdrojový kód: 02_Promenne_Prezentace

```
#include <stdio.h>
#define KONSTANTA 10

int main()
{
    printf("%d", KONSTANTA);
}
```

(Při použití konstanty `#define` nezapomeňte, že zápis syntaxe je trochu jiný, než u běžné proměnné tzn. bez středníku a rovná se.)

Zdrojový kód: 03_Promenne_Prezentace

```
#include <stdio.h>
const int konstanta = 10;

int main()
{
    printf("%d", konstanta);
}
```

Čtení uživatelského vstupu

- Pokud chceme načíst hodnoty, které zadá uživatel, tak k tomu využijeme funkci **scanf()**.

```
int x;  
printf("Vlozte cislo: \n");  
scanf("%d", &x);  
printf("Zadane cislo: %d\n", x);
```

Zdrojový kód: 04_Promenne_Prezentace

(Zatím berte pouze jako fakt, že u funkce scanf je skoro vždy potřeba žádat o adresu proměnné. Jedná se totiž o složitější problematiku, která funguje na principu ukazatelů a k těm se dostaneme až později.)

1

Scanf()

Funkce scanf se skládá ze dvou částí (formátovací značka, adresa proměnné).

Formátovací značku určíme pomocí datového typu a adresu proměnné pomocí znaku &.



02 Ukázka – výpočet průměru

- Necháme uživatele zadat tři čísla a poté z těchto čísel vypočítáme průměr, ten poté vypíšeme a zaokrouhlíme na dvě desetinná místa.

```
int x,y,z;  
double prumer;  
printf("Vlozte cisla: \n");  
scanf("%d%d%d", &x, &y, &z);  
prumer = (x + y + z) / 3;  
printf("Prumer: %.2lf\n", prumer);
```

Zdrojový kód: 05_Promenne_Prezentace

(Pokud bychom chtěli výpis
zaokrouhlit například na tři desetinná
místa, tak stačí napsat formát %.3lf)

1

Scanf()

Pomocí funkce **scanf()** můžeme najednou načítat více než jednu hodnotu, stačí přidat tolik formátovacích značek, kolik potřebujete proměnných.



01 Cvičení – zdvojnásobení hodnoty

- Vytvořte jednoduchý program, který požádá uživatele o číslo, které následně zdvojnásobí a výsledek vypíše do konzole.
- Pokud uživatel zadá například číslo 5, program by měl vypočítat $5 * 2 = 10$ a výsledek 10 zobrazit v konzoli.

```
Vlozte cislo k zdvojnásobení:  
5  
Vysledek: 10
```



01 Cvičení – zdvojnásobení hodnoty

```
int uzivatelsky_vstup;  
printf("Vlozte cislo k zdvojnásobení: \n");  
scanf("%d", &uzivatelsky_vstup);  
printf("Vysledek: %d\n", uzivatelsky_vstup * 2);
```

Zdrojový kód: 06_Promenne_Prezentace



02 Cvičení – jednoduchá kalkulačka

- Vaším úkolem je vytvořit jednoduchou kalkulačku, která bude komunikovat s uživatelem prostřednictvím konzole.
- Program nejprve požádá uživatele o zadání dvou čísel, která mohou být celá i desetinná. Poté program provede čtyři základní matematické operace: sčítání, odčítání, násobení a dělení.
- Výsledky všech těchto operací budou zaokrouhleny na dvě desetinná místa a zobrazeny v konzoli.

```
Zadejte dve cisla:  
1 2  
Soucet: 3.00  
Rozdil: -1.00  
Soucin: 2.00  
Podil: 0.50
```



02 Cvičení – jednoduchá kalkulačka

```
float x, y;  
printf("Zadejte dve cisla: \n");  
scanf("%f%f", &x, &y);  
printf("Soucet: %.2f\n", x + y);  
printf("Rozdil: %.2f\n", x - y);  
printf("Soucin: %.2f\n", x * y);  
printf("Podil: %.2f\n", x / y);
```

Zdrojový kód: 07_Promenne_Prezentace